

mars - a CLI tool

Mario Fernández

September 11, 2026

Contents

1	Introduction	2
1.1	About the author	2
1.2	What's <i>mars</i> ?	2
1.3	Intended Audiences	2
1.4	Scope	2
1.5	Technologies	3
2	Overall Description	4
2.1	Product Perspective	4
2.2	Product Functions	4
2.3	User Classes and Characteristics	4
2.4	Operating Environment	4
2.5	Use Case Diagram	5
3	Requirements	6
3.1	Functional Requirements	6
3.2	Non Functional Requirements	6
3.2.1	Usability Requirements	6
3.2.2	Performance Requirements	6
3.2.3	Software Requirements	6
3.2.4	Hardware Requirements	6
4	Implementation	7
4.1	File Structure	7
4.2	Commands	8
4.2.1	Welcome	8
4.2.2	Mars	8
4.2.3	String	8
4.2.4	Copy	9
4.2.5	Paste	9
5	Final Considerations	10
5.1	Closing thoughts	10

Chapter 1

Introduction

1.1 About the author

The entirety of the project, including its design, planning and implementation is maintained by the author, @mariolfh on Github [3]. As a computer science graduate, getting into contact with all sort of new concepts and ideas is something marvelous. So, the hopes of having several important and key tools in one place and at hand, mixed with a love for terminals, made the original thought that would soon evolve into this project.

1.2 What's *mars*?

Mars is a command-line interface tool designed to assist developers and power users with daily tasks right from the terminal. It serves as a personal tool-set that runs entirely offline, focusing on simplicity. This project follows three main objectives:

1. To learn and practice new technologies.
2. To share this tool and its advantages with the world, along with its code.
3. To showcase my abilities with certain preferred technologies.

1.3 Intended Audiences

This product is thought and designed for power users, which are users that have a certain level of technical knowledge, which include using the Command Prompt, installing *mars* and calling it with the corresponding commands. Therefore, the term 'power user' can range between computer hobbyists, scientists, developers and any other user with curiosity for programs beyond graphical user interfaces.

1.4 Scope

This program means to bring several most common functionalities into the command line interface of users who prefer its use over graphical user interfaces. Meant to assist in a simple way while staying local and offline, *mars* stands as the base for more programs of its type.

1.5 Technologies

The project was envisioned to be low maintenance and simple, something great for the command line to do. For this task, a low level language was needed, so in this case, Rust [5] was the best option. Both a personal selection and an unfamiliar technology, it proved worthy of the task. For the documentation you're reading, L^AT_EX [4] came along, as it's widely used in academia and was something I wanted to learn since college. Finally but not least, this is all being saved by a version control system, which in this case is Git [2].

Chapter 2

Overall Description

2.1 Product Perspective

This piece of software began from a crave the author had during college. On the average developer's day to day work flow filled with tools and graphical user interfaces, getting focus can be difficult, and having the right tools for the job can very often be a dread. With that in mind, the author intends to present a simple, offline, always accessible, terminal based and tool rich program that can satisfy power users' daily needs.

2.2 Product Functions

With the goal to create something power users enjoy, and thinking on designing functionalities that can be used on a daily basis locally, offline and with simplicity, the functions this program possess are:

- String functions: From uppercase, lowercase and randomize, most used string functions are available for use right from the terminal.
- Math functions: Adding and subtracting numbers is kind of basic, but it's integrated. Looking to dive into more complex mathematics with time.
- Logging information: without having to leave the terminal, you can log your thoughts or your notes of the day.
- Reminders: having a hard time remembering to drink water? let *mars* remind you!

2.3 User Classes and Characteristics

Power user	Users who know and love the terminal, using it in a regular basis
Developers	Users who use the terminal daily

2.4 Operating Environment

The program will operate on Microsoft Windows with support for other operating systems in the future. For now testing is being done only on Windows 10, so this is the platform recommended

to run the program in the near future.

2.5 Use Case Diagram

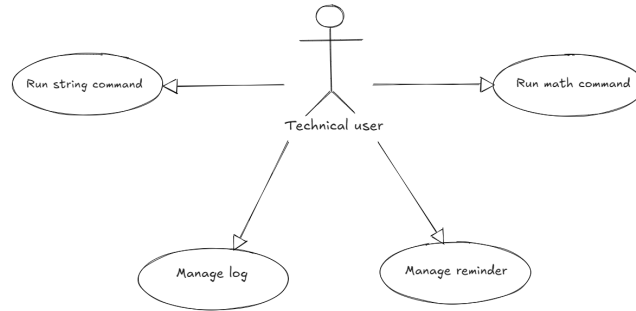


Figure 1: System's Use Case Diagram (Made by the author[3])

Chapter 3

Requirements

3.1 Functional Requirements

No.	Name	Description
1.	Print welcome message	The system shall print a welcome message to the user when running the command.
2.	Print inspirational phrase	The system shall print the inspirational phrase that motivated the development of <i>mars</i>
3.	Print uppercase string	The system shall print a given string in uppercase.
4.	Print lowercase string	The system shall print a given string in lowercase.

3.2 Non Functional Requirements

3.2.1 Usability Requirements

1. Accessibility: The system must be accessible from the terminal.

3.2.2 Performance Requirements

2. Response time: The system must return its responses in less than 100ms (milliseconds).

3.2.3 Software Requirements

3. Platform: The system must run on Microsoft Windows 10.

3.2.4 Hardware Requirements

4. Processor: The system must be run on a computer with a Dual - Core processor or better.

Chapter 4

Implementation

4.1 File Structure

This is the representation of the file structure used in the project. It contains the folders and files that are being published on GitHub and are related to the correct functioning of the application. It contains two folders, one containing all documentation related files and the other the code files.

```
documentation
├── Project Documentation.bib
├── Project Documentation.pdf
└── Project Documentation.tex
src
├── clipboard.rs
├── fileops.rs
├── main.rs
└── Cargo.toml
```

- ‘Project Documentation.bib’: This file contains the bibliographical information used in the documentation. All the sources I stored on Zotero are automatically exported to this file to then be cited in this document.
- ‘Project Documentation.pdf’: Final document file for the project, rendered in a standard digital document format.
- ‘Project Documentation.tex’: Raw code for the document to be rendered of. It included $\text{\TeX}$ and $\text{\LaTeX}$ code that generates the document.
- ‘clipboard.rs’: File storing the code that manages the clipboard operations, which are copy and paste.
- ‘fileops.rs’: File storing the code that takes care of the file operations, such as getting the file, setting values in the file and checking if the file exists.
- ‘main.rs’: Main file of the project which manages the code that understands the commands given and what to do with the arguments provided.
- ‘Cargo.toml’: Information file that stores the libraries that the project uses along with it’s versions.

4.2 Commands

This section goes deep into the usage of *mars* [1], explaining the main functionalities along with code snippets as demonstration. An important thing to notice is that *mars* works with the following structure:

```
// Basic command call
$ mars <command>
```

In some specific cases, the command will require one or more arguments to work properly. These arguments are required for operation and will be specified as needed in this document.

4.2.1 Welcome

This function prints a welcome message to the user. Firstly thought as a meme or joke, it stayed as one of the first functions to be written for the app. It also served as a test run to check if the program worked at all.

Usage

```
// Basic command call
$ mars welcome
Welcome to Mars!
```

4.2.2 Mars

This function prints the main inspirational quote for the *mars* project, said by Buzz Aldrin, an American pilot and astronaut. It's all there, at reach, waiting for us to get there, no matter how much time it takes, you just gotta make the effort to get there. It's very inspiring to myself.

Usage

```
// Basic command call
$ mars mars
'Mars is there, waiting to be reached.' - Buzz Aldrin, American pilot and astronaut,
2009
```

4.2.3 String

The command string works with two arguments: one indicating the operation to be performed and the other to specify the value that is used in said operation. Commands like this that use a value can work with the *** placeholder, making the program use the latest stored value for the task instead of a supplied value.

```
// Basic command call
$ mars string <operation> <value>
$ mars string <operation> *
```

Uppercase

This function requires a string as a parameter. This string is taken by the tool and transforms it into a fully uppercase string. This is very useful to myself, I've needed several times to work with titles in uppercase that were so long to type out again in lowercase, so this is here just for that.

```
// Basic command call
$ mars string uppercase <string>
<STRING>
```

Lowercase

This function requires a string as a parameter. This string is taken by the tool and is returned with all characters in lowercase. This saved lots of work during college, in which I needed to present author names in lowercase for my papers. Back then it was written in Python and just sitting around in a scripts folder, but now it's here as a part of something much bigger.

```
// Basic command call
$ mars string lowercase <STRING>
<string>
```

Size

This function requires a string as a parameter. The string's length in characters is then calculated and returned.

```
// Basic command call
$ mars string sizee <STRING>
8
```

4.2.4 Copy

This function doesn't require any parameters to work. It takes the latest value that was processed with Mars and inserts it on the system's clipboard to be used across the system.

```
// Basic command call
$ mars copy
Last used element copied from Mars to the clipboard.
```

4.2.5 Paste

This function doesn't require any parameters to work. It takes whatever the system's clipboard has in store and makes it available inside Mars for later use through the * placeholder.

```
// Basic command call
$ mars paste
Clipboard contents pasted into Mars from the clipboard.
```

Chapter 5

Final Considerations

5.1 Closing thoughts

I wanted to take the time to express myself down here, and so, i want to thank several people who i won't mention for believing in me and for boosting to come back into my field and heart. Also, I want you, the reader, to take this as a sign that something bright and big is out there for you, even if you think it hasn't come or it isn't your turn yet. Guess what? You make your turn! I did the same thing with this project, my first project, on my own. Doing it, scared and excited, but doing it. Go after the things you like and want to know. The only thing stopping you is yourself.

Citations

- [1] Mario Fernández. *Mariolfh/Mars*. May 16, 2025. URL: <https://github.com/mariolfh/mars> (visited on 05/17/2025).
- [2] *Git*. URL: <https://git-scm.com/> (visited on 05/17/2025).
- [3] GitHub, Inc. *Mariolfh - Overview*. Github. 2025. URL: <https://github.com/mariolfh>.
- [4] *LaTeX - A Document Preparation System*. URL: <https://www.latex-project.org/> (visited on 05/17/2025).
- [5] *Rust Programming Language*. URL: <https://www.rust-lang.org/> (visited on 05/17/2025).